

Deep Learning

3. Perceptrons Multi-Couches

Romain Tavenard

Limites du Perceptron

- Entrée : $x = \{x_0, \dots, x_D\}$
- Sortie :
$$a = \varphi\left(\sum_j w_j x_j + b\right)$$
- Peut faire
 - Régression linéaire
 - Frontières linéaires pour la classification
 - Sortie unique

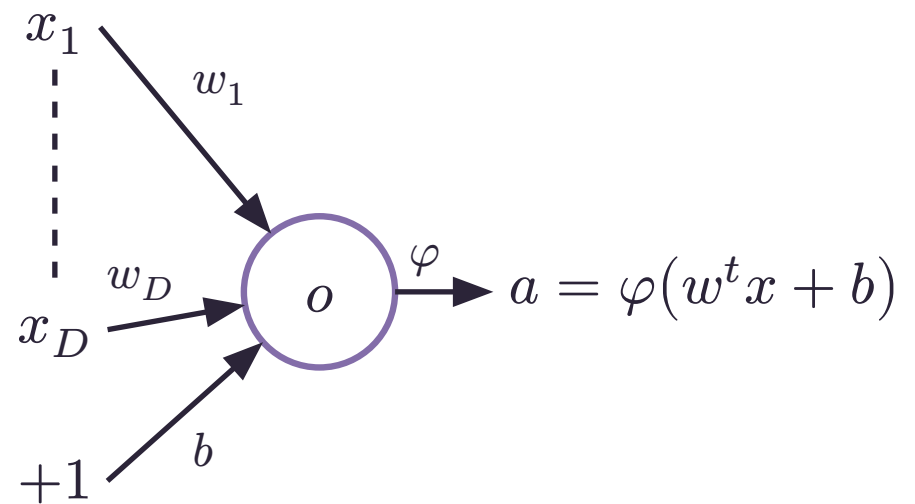
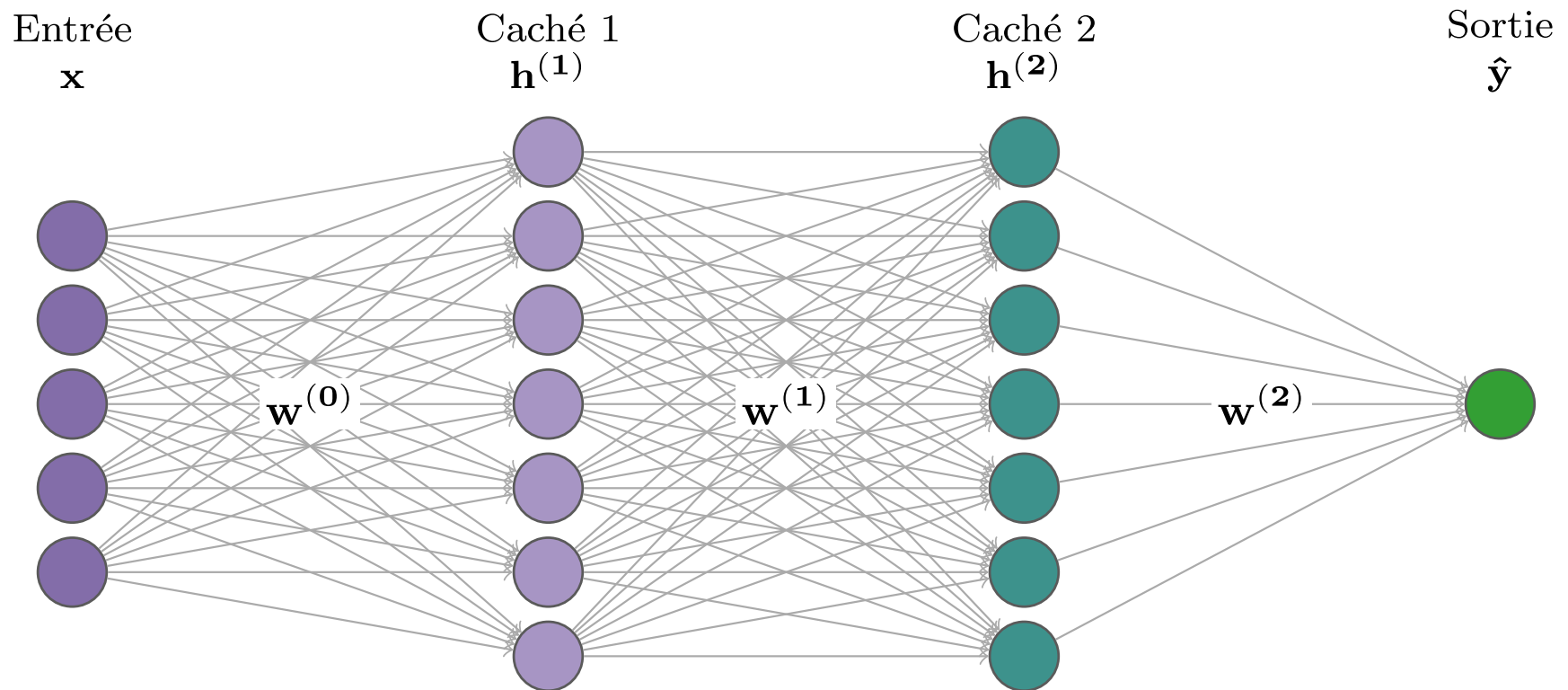


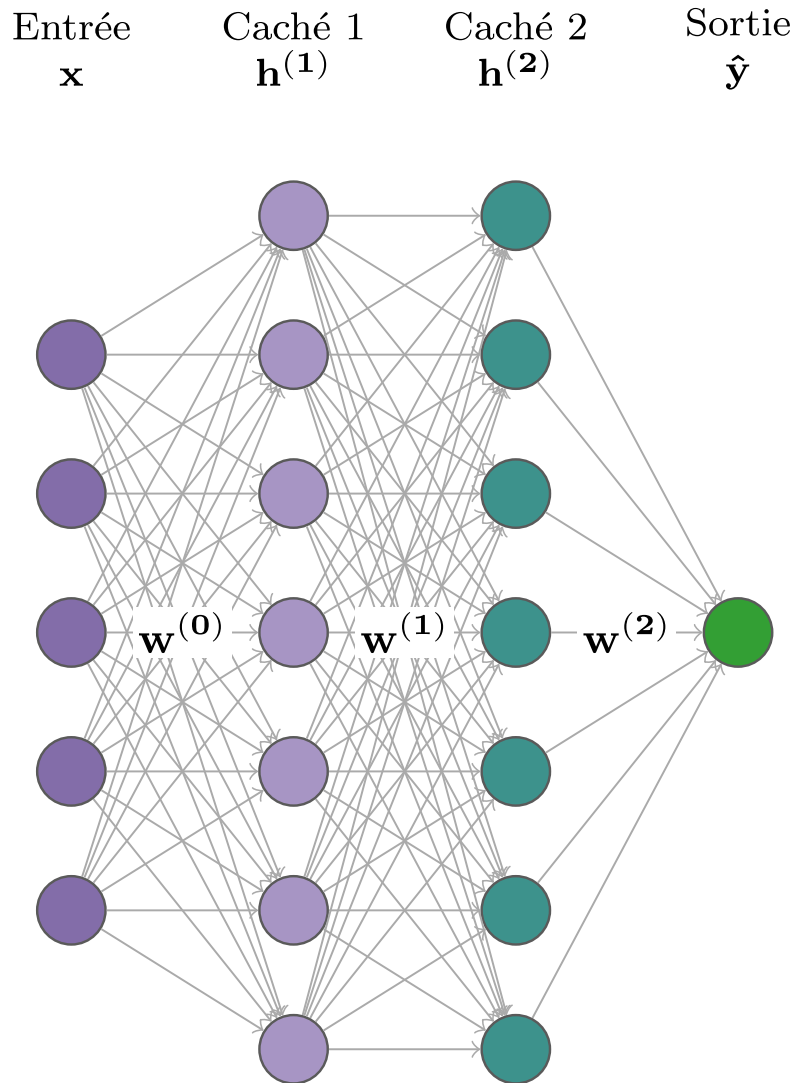
Figure : schéma du Perceptron

Modèle du Perceptron Multi-Couches (MLP)

(Rumelhart, Hinton & Williams, 1985)



Modèle du Perceptron Multi-Couches (MLP) (Rumelhart, Hinton & Williams, 1985)



$$\hat{y} = \varphi_{\text{out}} \left(\sum_i w_i^{(2)} h_i^{(2)} + b^{(2)} \right)$$

$$\forall i, h_i^{(2)} = \varphi \left(\sum_j w_{ij}^{(1)} h_j^{(1)} + b_i^{(1)} \right)$$

$$\forall i, h_i^{(1)} = \varphi \left(\sum_j w_{ij}^{(0)} x_j + b_i^{(0)} \right)$$

Pourquoi introduire des couches cachées ?

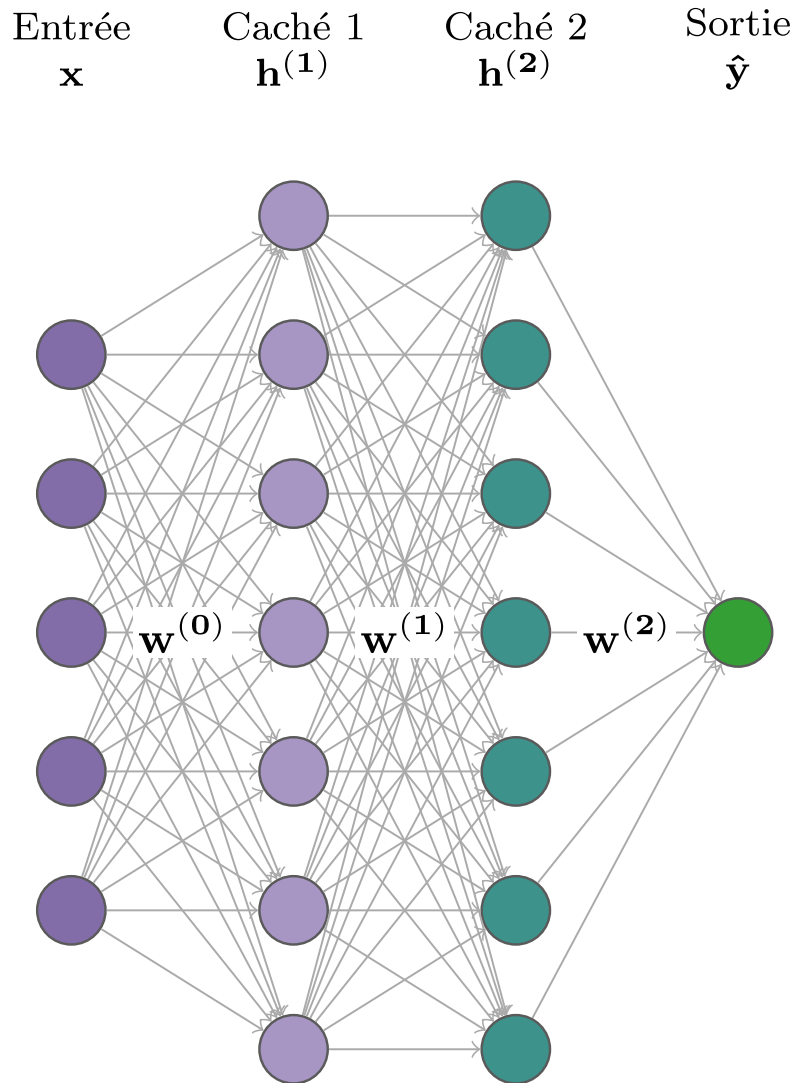
- **Intuition** : les couches cachées permettent au réseau d'apprendre des transformations non linéaires de l'entrée — elles remodelent les données dans un espace où la couche de sortie (linéaire) peut résoudre la tâche
- **Résultat théorique** (théorème d'approximation universelle, Cybenko, 1989) : une seule couche cachée, avec suffisamment de neurones, peut approximer **n'importe quelle** fonction raisonnable avec une précision arbitraire
- **À retenir en pratique** :
 - une couche cachée suffit **en théorie**
 - mais le nombre de neurones nécessaires peut être énorme

Pourquoi introduire des couches cachées ?

- ▶ en pratique, **empiler davantage de couches (plus petites)** est plus efficace et généralise mieux

Briques de base des réseaux de neurones

Couches d'entrée/sortie



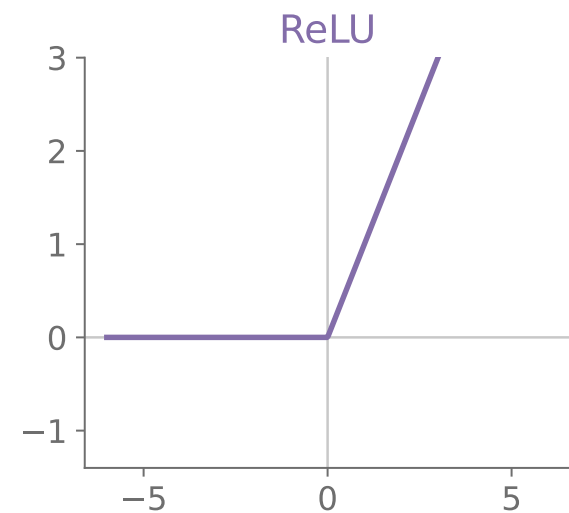
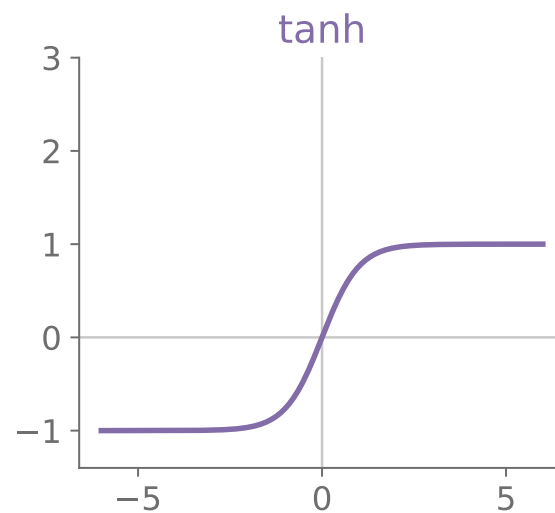
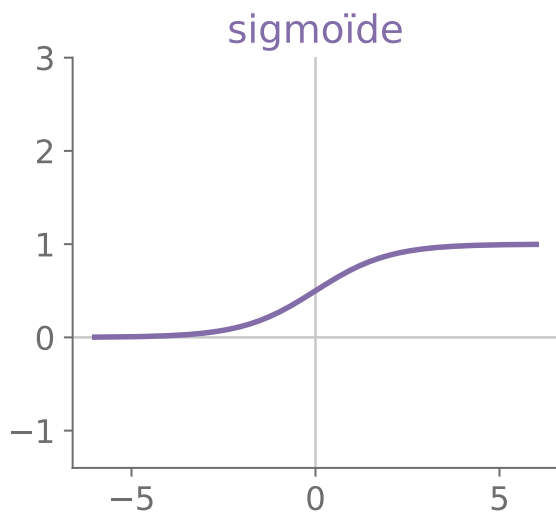
Étant donné un jeu de données (X, y)

- Contraintes sur la structure du modèle :
 - La dimension de la couche d'entrée est le nombre de caractéristiques dans X
 - La couche de sortie possède autant d'unités que de colonnes dans y

Briques de base des réseaux de neurones

Fonctions d'activation

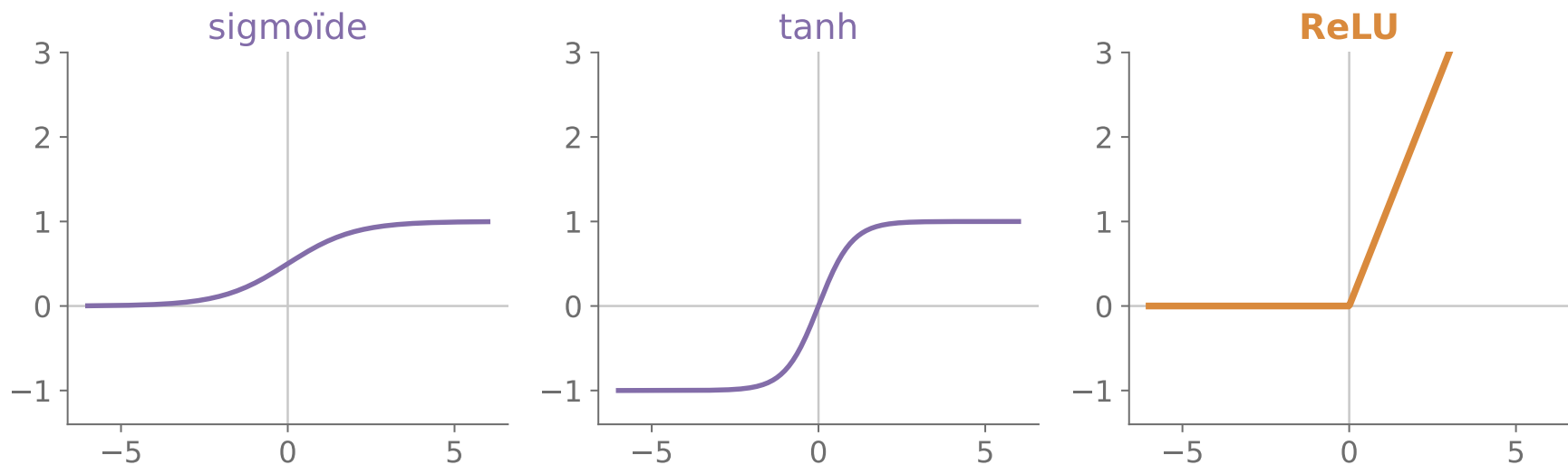
- Propriétés importantes
 - φ doit être dérivable presque partout
 - Non-linéarités
 - Un régime quasi linéaire
- Exemples



Briques de base des réseaux de neurones

Fonctions d'activation : ReLU domine

- ReLU est devenue le choix par défaut **pour les couches internes**
- 2 raisons principales :
 - peu coûteuse à calculer (ReLU et sa dérivée)
 - disparition des gradients (davantage de détails plus tard)



ReLU est aujourd'hui le choix par défaut pour les couches internes

Briques de base des réseaux de neurones

Fonctions d'activation : le cas de la couche de sortie

- Les fonctions d'activation de sortie déterminent les valeurs :
 - identité (“linear” dans keras) : tout réel
 - ReLU : toute valeur positive
 - sigmoïde : toute valeur dans $[0, 1]$
 - softmax : >0 et somme à 1 (sur les neurones de sortie)

